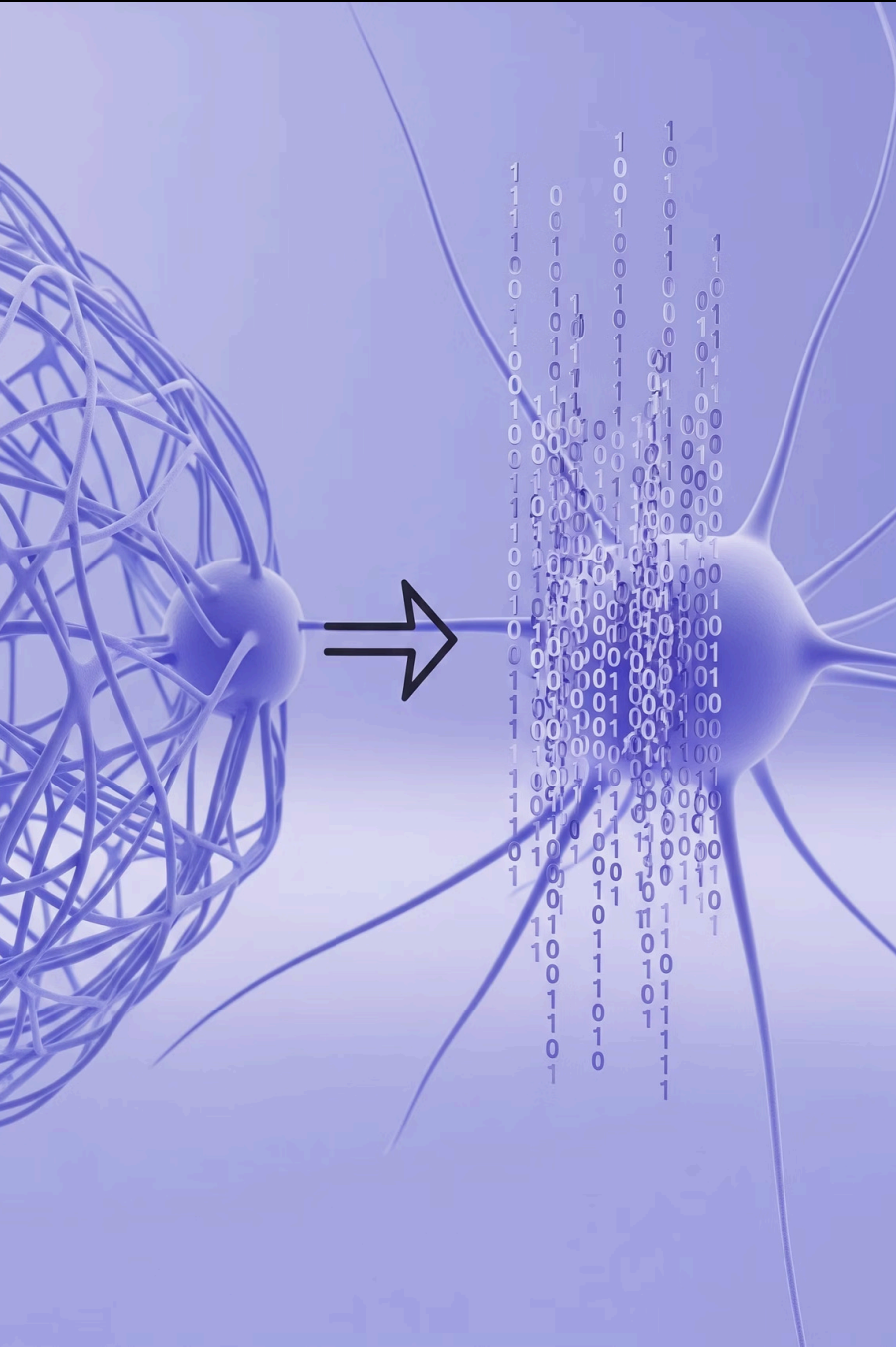




# Quantization-Aware Training (QAT)

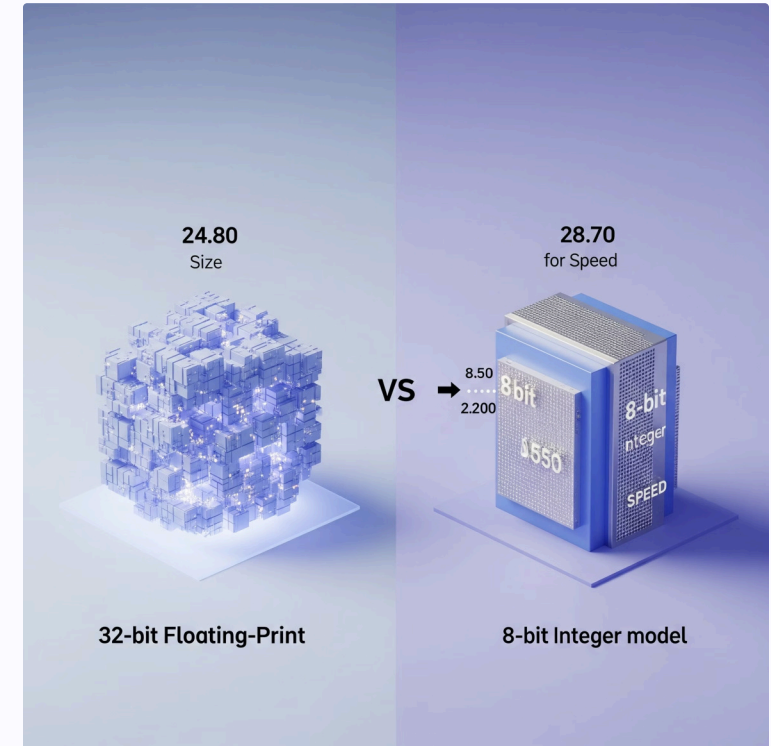
Make models lighter & faster by training with quantization effects simulated, resulting in deployment-ready solutions for edge & mobile applications.

# Why Quantization?

Traditional deep learning models use **32-bit floating point (FP32)** precision, resulting in:

- Large memory footprint
- High computational requirements
- Deployment challenges on resource-constrained devices

**Quantization** represents weights & activations in **8-bit integers (INT8)**, dramatically reducing resource requirements while maintaining performance.



Goal: Lower size + latency with minimal accuracy loss



# Post-Training Quantization vs QAT

## Post-Training Quantization (PTQ)

- Convert FP32  $\rightarrow$  INT8 *after* training
- Fast implementation process
- Minimal engineering effort
- Can result in significant accuracy drops
- No model adaptation to quantization noise

## Quantization-Aware Training (QAT)

- Train model *with* quantization simulation
- Learns to adapt during training process
- Maintains higher accuracy than PTQ
- Recovers from quantization-induced errors
- Results in deployment-ready models

# How QAT Works



## Training Phase

- Maintain master weights in FP32
- Insert fake quantization operations
- Simulate rounding effects to INT8
- Forward pass: quantize → compute → dequantize
- Backward pass: learn to compensate for quantization noise



## Inference Phase

- Deploy actual INT8 quantized model
- Remove training-only operations
- Use hardware-optimized INT8 operations
- Dramatically reduced memory footprint
- Significantly faster computation

**Result:** Model becomes **robust to quantization noise** through adaptation during training



# Benefits of QAT

4x

## Memory Reduction

Storing weights in INT8 format requires 75% less memory than FP32 models

3x

## Inference Speedup

Faster computation on CPUs, TPUs, and specialized edge hardware

~0%

## Accuracy Drop

Minimal accuracy degradation compared to FP32 models with proper QAT

10x

## Energy Efficiency

Lower power consumption enables on-device AI in battery-constrained environments

# TensorFlow Example – QAT

```
import tensorflow_model_optimization as tfmot

# Create quantization-aware model
qat_model = tfmot.quantization.keras.quantize_model(model)

# Compile and train as usual
qat_model.compile(optimizer='adam',
                  loss='categorical_crossentropy')
qat_model.fit(train_data, train_labels, epochs=5)

# The resulting model can be converted to TFLite
converter = tf.lite.TFLiteConverter.from_keras_model(qat_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
quantized_tflite_model = converter.convert()
```

## Key Implementation Points:

- `quantize_model()` wraps layers with fake quantization operations
- Model trains with quantization effects simulated in forward pass
- Gradients flow through the fake quantization operations
- Model learns to compensate for quantization noise
- Final TFLite conversion creates deployment-ready INT8 model

# PyTorch Example – QAT

```
import torch.quantization as tq

# Create model and set quantization config
model_fp32 = Net()
model_fp32.qconfig = tq.get_default_qat_qconfig('fbgemm')

# Prepare model for QAT
model_prepared = tq.prepare_qat(model_fp32)

# Train with QAT
train(model_prepared, data_loader)

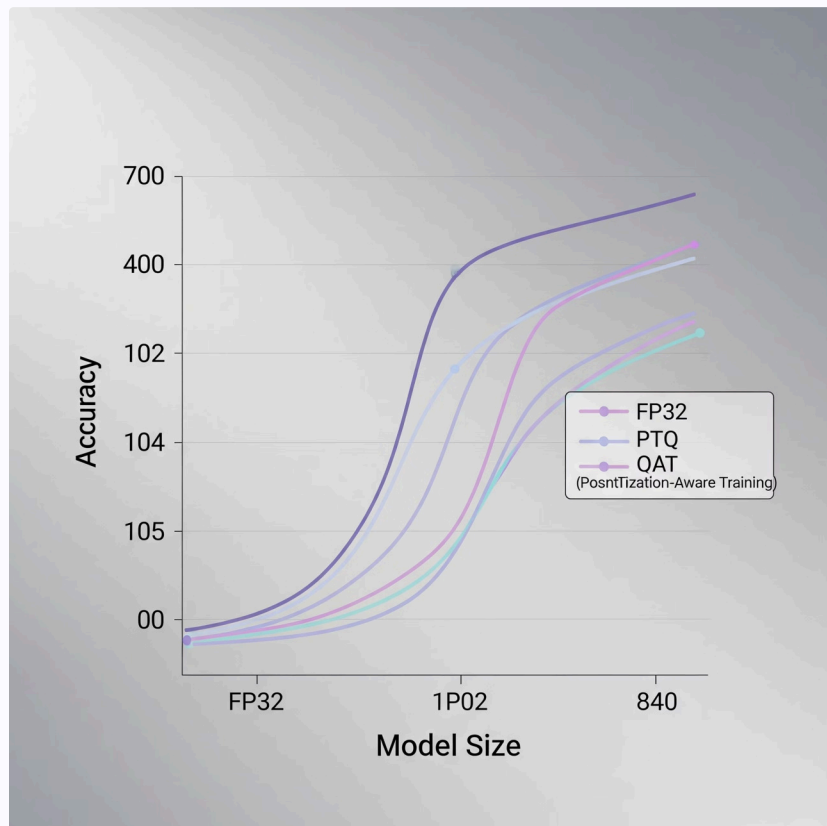
# Convert to deployable quantized model
model_int8 = tq.convert(model_prepared)

# Save the quantized model
torch.jit.save(torch.jit.script(model_int8), "quantized_model.pt")
```

## Key Implementation Points:

- `qconfig` defines quantization parameters (bit width, observers)
- `prepare_qat()` inserts fake quantization operations
- Training occurs with simulated quantization effects
- `convert()` transforms to actual INT8 model
- Different backends available ('fbgemm' for server, 'qnnpack' for mobile)

# Accuracy vs Efficiency Tradeoff



## Key Considerations

The effectiveness of quantization varies by:

- **Model architecture** - larger models (BERT, ResNet) typically see better benefits from QAT
- **Task complexity** - simpler tasks may be satisfied with PTQ
- **Precision requirements** - critical applications may need higher precision



Always evaluate accuracy metrics after quantization!

# Hardware Impact



## CPUs

- Intel AVX512 VNNI instructions
- ARM NEON SIMD optimizations
- 2-4x throughput improvement with INT8



## Mobile/Edge

- Qualcomm Hexagon DSP
- Apple Neural Engine
- MediaTek APU
- Optimized for INT8 operations



## TPUs

- Native INT8 matrix multiplication
- Massive throughput improvement
- Google Edge TPU require quantization



## GPUs

- TensorRT INT8 optimizations
- Smaller gains than other hardware
- Still excel at FP16/FP32 operations

Hardware-specific optimizations are key to unlocking maximum performance gains from quantization

# Industry Applications



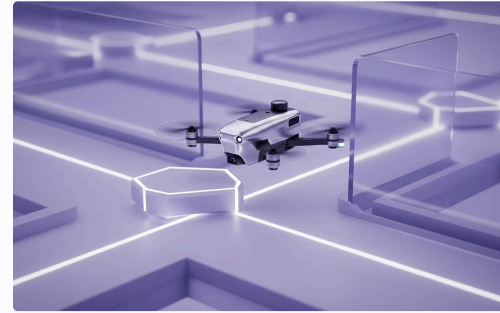
## Mobile Applications

- On-device speech recognition
- Real-time camera effects
- Offline language translation
- AR features with object detection



## IoT Devices

- Anomaly detection in sensors
- Voice assistants in smart homes
- Predictive maintenance
- Ultra-low power applications



## Autonomous Systems

- Real-time object detection
- Autonomous navigation
- Gesture recognition
- Battery-efficient inference



## Cloud Deployment

- Reduced inference costs
- Higher throughput per server
- Lower power consumption
- Scaled deployment savings

# Limitations of QAT

## Technical Challenges

### Training Complexity

More complex training pipeline than standard training, requiring specialized knowledge and tools

### Increased Training Time

QAT typically requires 1.5-2x longer training time than standard FP32 training

### Limited Operation Support

Not all operations are easily quantizable (custom layers, certain activation functions)

## Workflow Considerations

- **Integration complexity** with existing ML pipelines
- **Hardware-specific** quantization requirements
- **Debuggability challenges** with quantization issues
- **Model architecture limitations** that may prevent effective quantization

 **Try PTQ first!** Only move to QAT if accuracy drops are unacceptable with PTQ.



# Summary: Quantization-Aware Training

- 1 Quantization shrinks models from FP32 to INT8**  
Dramatically reducing memory requirements and computational needs
- 2 QAT simulates quantization during training**  
Allowing the model to adapt to quantization effects and maintain accuracy
- 3 Results: Smaller, faster, accurate models**  
Enabling AI deployment on resource-constrained devices without significant performance loss
- 4 Essential for edge & mobile AI deployment**  
Unlocking new use cases where computational resources are limited

QAT bridges the gap between high-performance models and efficient deployment